

Chapitre 4 : Fonctions avancées

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-23

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) écrire des fonctions avec des paramètres par défaut et les appeler avec des arguments nommés ; 2) éviter le piège du paramètre par défaut modifiable ; 3) accepter un nombre variable d'arguments (`*args` , `**kwargs`) et déballer une liste ou un dictionnaire à l'appel ; 4) expliquer la **portée** des variables ; 5) passer une fonction en paramètre et en renvoyer une ; 6) utiliser `lambda` , `map` et `filter` ; 7) écrire des compréhensions de listes, de dictionnaires et d'ensembles ; 8) écrire une fonction **récursive**.
Prérequis	Chapitres 1 à 3 ; fonctions de L1 (`def` , paramètres, `return`)
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (atelier compris)
Matériel	Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code ; le registre complet (atelier 3)
Livrable	L'atelier 4 : les filtres et les rapports à la demande, publiés sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch4` et copiez-y le fichier `donnees.py` du chapitre 3 (les 7 livraisons) : plusieurs exemples l'importent.

---

1. Des fonctions plus souples

En L1, une fonction recevait ses arguments **dans l'ordre**, tous obligatoires. Dès qu'une fonction a beaucoup de réglages, ça devient pénible et dangereux : qui se souvient que le troisième argument est le supplément pour colis lourd ? Ce chapitre donne les outils pour écrire des fonctions **souples, lisibles et réutilisables** : c'est ce qui permet de construire, dans l'atelier, des rapports « à la demande » en quelques lignes.

2. Paramètres par défaut et arguments nommés

2.1 Les valeurs par défaut

Créez `essai1.py` :

```
TARIFS = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000, "Parcelles": 2000, "Pikine": 2500}


def frais(quartier, poids, express=False, supplement_lourd=500):
    montant = TARIFS[quartier]
    if poids > 5:
        montant += supplement_lourd
    if express:
        montant *= 2
    return montant


print(frais("Yoff", 2))
print(frais("Yoff", 7))
print(frais("Yoff", 7, express=True))
print(frais("Yoff", 7, supplement_lourd=1000))
print(frais(poids=7, quartier="Pikine"))
```

```
2000
2500
5000
3000
3000
```

Morceau	Ce qu'il veut dire
`express=False`	Un paramètre **par défaut** : si l'appel ne le donne pas, il vaut `False`
`frais("Yoff", 2)`	Seuls les deux paramètres obligatoires ; `express` et `supplement_lourd` prennent leur valeur par défaut
`frais("Yoff", 7, express=True)`	Un argument **nommé** : on dit à quel paramètre il correspond. Plus lisible que `frais("Yoff", 7, True)`
`frais("Yoff", 7, supplement_lourd=1000)`	On peut sauter `express` (il garde sa valeur par défaut) et ne donner que `supplement_lourd`

Morceau	Ce qu'il veut dire
`frais(poids=7, quartier="Pikine")`	Avec des noms, l'ordre n'a plus d'importance

Règle : dans la définition, les paramètres par défaut viennent **après** les obligatoires. `def f(a=1, b):` donne `SyntaxError: parameter without a default follows parameter with a default`.

> 🪛 **Essayez** : appelez `frais("Yoff")`, puis `frais("Yoff", 7, False, 500, "en plus")`, puis `frais("Yoff", 7, rapide=True)`. Messages : `frais() missing 1 required positional argument: 'poids'` (« il manque l'argument obligatoire `poids` ») ; `frais() takes from 2 to 4 positional arguments but 5 were given` (« de 2 à 4 arguments attendus, 5 reçus ») ; `frais() got an unexpected keyword argument 'rapide'` (« argument nommé inattendu »). Lisez toujours ces messages : ils disent exactement ce qui ne va pas. Annulez.

2.2 Le piège de la valeur par défaut modifiable

Créez `essai2.py` :

```
def ajouter(numero, liste=[]):
    liste.append(numero)
    return liste


print(ajouter(1))
print(ajouter(2))


def ajouter_bien(numero, liste=None):
    if liste is None:
        liste = []
    liste.append(numero)
    return liste


print(ajouter_bien(1))
print(ajouter_bien(2))
```

```
[1]
[1, 2]
[1]
[2]
```

Le deuxième appel de `ajouter` renvoie `[1, 2]` : la liste du premier appel a **survécu** ! La valeur par défaut `[]` est créée **une seule fois**, quand Python lit la définition de la fonction, et **réutilisée** à chaque appel. Comme `append` la modifie, elle grossit d'un appel à l'autre.

Règle : jamais de liste, de dictionnaire ou d'ensemble comme valeur par défaut. Utilisez `None`, et créez l'objet dans la fonction (`ajouter_bien`). `is None` teste si la valeur est exactement `None`.

3. Un nombre variable d'arguments

3.1 `*args` et `**kwargs`

Créez `essai3.py` :

```
def total(*montants):
    print("reçu :", montants)
    return sum(montants)


print(total(1000, 2500, 2000))
print(total())


def fiche(numero, **infos):
    print("numéro", numero, "et", infos)


fiche(8, livreur="Ibou", quartier="Parcelles")
fiche(9)
```

```
reçu : (1000, 2500, 2000)
5500
reçu : ()
0
numéro 8 et {'livreur': 'Ibou', 'quartier': 'Parcelles'}
numéro 9 et {}
```

Morceau	Ce qu'il veut dire
`*montants`	« tous les arguments **positionnels** restants » ; ils arrivent dans un **tuple**
`total()`	Aucun argument : le tuple est vide, la somme vaut 0
`**infos`	« tous les arguments **nommés** restants » ; ils arrivent dans un **dictionnaire**

Les noms `args` et `kwargs` (*keyword arguments*) sont une convention : ce sont les étoiles qui comptent. On choisit souvent des noms parlants, comme `*montants`.

3.2 Déballer à l'appel

Les étoiles servent aussi **dans l'autre sens** : pour déballer une liste ou un dictionnaire en arguments. Créez `essai4.py` :

```python
def frais(quartier, poids, express=False):
    return f"{quartier}, {poids} kg, express : {express}"


arguments = ["Plateau", 3]
options = {"express": True}
print(frais(*arguments))
print(frais(*arguments, **options))
livraison = {"quartier": "Médina", "poids": 1.5}
print(frais(**livraison))
```

```
Plateau, 3 kg, express : False
Plateau, 3 kg, express : True
Médina, 1.5 kg, express : False
```

`frais(*arguments)` équivaut à `frais("Plateau", 3)` ; `frais(**livraison)` à `frais(quartier="Médina", poids=1.5)`. Pratique quand les arguments viennent d'un fichier ou d'une autre fonction.

4. La portée des variables

Une variable créée **dans** une fonction n'existe que dans cette fonction : c'est une variable **locale**. Créez `essai5.py` :

```python
SEUIL = 45


def est_en_retard(duree):
    return duree > SEUIL


def compter_retards(durees):
    nombre = 0
    for d in durees:
        if est_en_retard(d):
            nombre += 1
    return nombre


print(compter_retards([28, 65, 41, 58]))
try:
    print(nombre)
except NameError as e:
    print("Erreur :", e)

compteur = 0


def incrementer():
    compteur += 1


try:
    incrementer()
except UnboundLocalError as e:
    print("Erreur :", e)
```

```
2
Erreur : name 'nombre' is not defined
Erreur : cannot access local variable 'compteur' where it is not associated with a value
```

Ce qu'on fait	Ce qui se passe
Lire une variable du programme (`SEUIL`) dans une fonction	Autorisé : Python la cherche d'abord dans la fonction, puis à l'extérieur
Utiliser `nombre` **hors** de `compter_retards`	`NameError` : la variable locale a disparu à la fin de la fonction
Modifier une variable du programme (`compteur += 1`) dans une fonction	`UnboundLocalError` : dès qu'une fonction **affecte** une variable, Python la considère comme locale ; or elle n'a pas encore de valeur locale

Python cherche un nom dans cet ordre : **L**ocal (la fonction), **E**nglobant (une fonction qui contient la fonction, section 6.3), **G**lobal (le fichier), puis les noms **B**uilt-in de Python (`print` , `len` ...). C'est la règle **LEGB**.

> 🔵 **Standard : et `global` ?** Écrire `global compteur` dans la fonction permettrait de modifier la variable globale. **Évitez-le** : une fonction qui modifie des variables extérieures est difficile à comprendre et à tester. Préférez une fonction qui **reçoit** une valeur et **renvoie** le résultat : `compteur = incrementer(compteur)`. Les **constantes** globales en majuscules (`SEUIL` , `TARIFS`), elles, sont lues partout sans problème.

5. Les fonctions sont des valeurs

5.1 Passer une fonction en paramètre

En Python, une fonction est une **valeur** comme une autre : on peut la ranger dans une variable, la mettre dans une liste, la passer à une autre fonction. Créez `essai6.py` :

```
from donnees import LIVRAISONS


def appliquer(livraisons, regle):
    return [l for l in livraisons if regle(l)]


def est_lourd(livraison):
    return livraison["poids"] > 5


def est_a_pikine(livraison):
    return livraison["quartier"] == "Pikine"


print([l["numero"] for l in appliquer(LIVRAISONS, est_lourd)])
print([l["numero"] for l in appliquer(LIVRAISONS, est_a_pikine)])
print(est_lourd, type(est_lourd))
```

```
[2, 5, 7]
[2, 6]
<function est_lourd at 0x0000023BDF51C360>
<class 'function'>
```

Morceau	Ce qu'il veut dire
`appliquer(livraisons, regle)`	Une fonction qui reçoit **une autre fonction**, `regle` , et l'appelle sur chaque livraison
`appliquer(LIVRAISONS, est_lourd)`	On passe la fonction **sans parenthèses** : la fonction elle-même, pas son résultat
`regle(l)`	Dans `appliquer` , on **appelle** la fonction reçue

C'est exactement ce que fait `key=` dans `sorted` (chapitre 3) : on donne à `sorted` une fonction qu'il appellera sur chaque élément.

> 🏏 **Essayez** : écrivez `appliquer(LIVRAISONS, est_lourd())`. Message : `TypeError: est_lourd() missing 1 required positional argument: 'livraison'` : avec les parenthèses, Python **appelle** `est_lourd` tout de suite, sans argument. Annulez.

5.2 `lambda`, `map` et `filter`

Pour une règle très courte, écrire une fonction avec `def` est lourd. `lambda` crée une petite fonction **sans nom**, sur une ligne. Créez `essai7.py` :

```
from donnees import LIVRAISONS

print([l["numero"] for l in filter(lambda l: l["duree_min"] > 45, LIVRAISONS)])
print(list(map(lambda l: l["frais"] * 2, LIVRAISONS)))
double = lambda x: x * 2
print(double(21))
print(sorted(LIVRAISONS, key=lambda l: l["poids"])[0]["numero"])
```

```
[2, 5, 6]
[2000, 6000, 4000, 2000, 4000, 5000, 5000]
42
4
```

Morceau	Ce qu'il veut dire
`lambda l: l["duree_min"] > 45`	Une fonction qui reçoit `l` et renvoie `l["duree_min"] > 45`. Équivaut à `def f(l): return l["duree_min"] > 45`
`filter(regle, liste)`	Garde les éléments pour lesquels `regle` renvoie `True`
`map(fonction, liste)`	Applique la fonction à chaque élément
`list(...)`	`map` et `filter` renvoient des objets « paresseux » (chapitre 5) : `list` les transforme en liste

Une `lambda` ne contient qu'**une expression** (pas de `if` sur plusieurs lignes, pas de boucle). `double = lambda x: x * 2` fonctionne, mais si l'on donne un nom, autant utiliser `def`, plus lisible.

6. Les compréhensions

6.1 Listes, dictionnaires, ensembles

`map` et `filter` s'écrivent souvent plus lisiblement avec une **compréhension**, que vous avez déjà croisée. Créez `essai8.py` :

```python
from donnees import LIVRAISONS

numeros_lourds = [l["numero"] for l in LIVRAISONS if l["poids"] > 5]
print(numeros_lourds)
etiquettes = ["lourd" if l["poids"] > 5 else "léger" for l in LIVRAISONS]
print(etiquettes)
frais_par_numero = {l["numero"]: l["frais"] for l in LIVRAISONS}
print(frais_par_numero)
livreurs = {l["livreur"] for l in LIVRAISONS}
print(sorted(livreurs))
grille = [(q, k) for q in ["Yoff", "Pikine"] for k in [1, 2]]
print(grille)
total = sum(l["frais"] for l in LIVRAISONS)
print(total)
```

```
[2, 5, 7]
['léger', 'lourd', 'léger', 'léger', 'lourd', 'léger', 'lourd']
{1: 1000, 2: 3000, 3: 2000, 4: 1000, 5: 2000, 6: 2500, 7: 2500}
['Awa', 'Fatou', 'Modou']
[('Yoff', 1), ('Yoff', 2), ('Pikine', 1), ('Pikine', 2)]
14000
```

Forme	Ce qu'elle produit
`[expression for x in source if condition]`	Une **liste** ; le `if` final **filtre**
`[a if condition else b for x in source]`	Une liste ; le `if ... else` **avant** le `for` **choisit** la valeur (rien n'est filtré)
`{cle: valeur for x in source}`	Un **dictionnaire**
`{expression for x in source}`	Un **ensemble**
`[... for a in A for b in B]`	Deux boucles imbriquées, dans l'ordre de lecture : pour chaque `a`, chaque `b`
`sum(expression for x in source)`	Une **expression génératrice** (sans crochets) : les valeurs sont produites une par une, sans créer de liste (chapitre 5)

> 🟢 **Débutant : compréhension ou boucle ?** Une compréhension est idéale pour **construire** une liste, un dictionnaire ou un ensemble en une ligne lisible. Si la logique demande plusieurs conditions, des `print` ou des `try`, écrivez une boucle normale : un code lisible vaut mieux qu'un code court.

6.2 Fonctions qui fabriquent des fonctions

Une fonction peut **créer** et **renvoyer** une autre fonction. Créez `essai10.py` :

```python
from donnees import LIVRAISONS


def fabriquer_filtre(seuil):
    def filtre(livraison):
        return livraison["duree_min"] > seuil
    return filtre


retard_45 = fabriquer_filtre(45)
retard_60 = fabriquer_filtre(60)
print([l["numero"] for l in LIVRAISONS if retard_45(l)])
print([l["numero"] for l in LIVRAISONS if retard_60(l)])
```

```
[2, 5, 6]
[2]
```

`fabriquer_filtre(45)` renvoie une fonction `filtre` qui **se souvient** de `seuil = 45`, même après la fin de `fabriquer_filtre` (c'est le **E** de LEGB, la fonction englobante). On appelle cela une **fermeture** (*closure*). C'est la base des **décorateurs** du chapitre 5, et l'outil central de l'atelier : des « fabriques » de filtres réglables.

7. La récursivité

Une fonction **récursive** s'appelle elle-même. C'est naturel pour les problèmes qui contiennent des versions plus petites d'eux-mêmes. Créez `essai9.py` :

```python
def factorielle(n):
    if n <= 1:
        return 1
    return n * factorielle(n - 1)


print(factorielle(5))

zones = {
    "Dakar": {
        "Plateau": 12,
        "Médina": 30,
        "Parcelles": {"Unité 10": 8, "Unité 17": 14},
    },
    "Pikine": {"Thiaroye": 21, "Guinaw Rail": 9},
}


def total_colis(zone):
    if isinstance(zone, int):
        return zone
    return sum(total_colis(sous_zone) for sous_zone in zone.values())


print(total_colis(zones))
print(total_colis(zones["Dakar"]))


def sans_fin(n):
    return sans_fin(n + 1)


try:
    sans_fin(0)
except RecursionError as e:
    print("Erreur :", e)
```

```
120
94
64
Erreur : maximum recursion depth exceeded
```

Une fonction récursive a toujours deux parties :

Partie	**Dans `factorielle`**	**Dans `total_colis`**
Le **cas de base** : la réponse est connue directement, on s'arrête	`n <= 1` : 1	La zone est un nombre (`isinstance(zone, int)`) : c'est le nombre de colis
Le **cas récursif** : on se ramène à un problème **plus petit**	`n * factorielle(n - 1)`	La somme des totaux des **sous-zones**

Déroulons `factorielle(5)` : 5 x `factorielle(4)` = 5 x 4 x `factorielle(3)` = 5 x 4 x 3 x 2 x `factorielle(1)` = 5 x 4 x 3 x 2 x 1 = 120.

`total_colis` est le vrai intérêt de la récursivité : la structure des zones peut avoir **autant de niveaux qu'on veut** (Parcelles est découpée en unités, une unité pourrait l'être en rues), et la même fonction de 4 lignes les parcourt toutes. Avec des boucles, il faudrait savoir à l'avance combien de niveaux imbriquer. Total : 12 + 30 + 8 + 14 + 21 + 9 = 94.

Sans cas de base (`sans_fin`), la fonction s'appelle indéfiniment ; Python l'arrête après environ 1 000 appels : `RecursionError: maximum recursion depth exceeded` .

Exercices

Créez un dossier `exercices-ch4` .

- ★ **Exercice 1 : le billet de car.** Écrivez `prix_billet(distance_km, classe="standard", enfant=False, prix_km=25)` : distance x prix au kilomètre, x 1,5 en classe « confort », divisé par 2 pour un enfant, arrondi. Attendu : `prix_billet(70)` 1 750 ; `prix_billet(70, "confort")` 2 625 ; `prix_billet(190, enfant=True)` 2 375 ; `prix_billet(distance_km=270, prix_km=20)` 5 400. (Section 2.)
- ★ **Exercice 2 : `*args` et `**kwargs` .** Écrivez `moyenne(*notes)` , qui lève une `ValueError` claire si aucune note n'est donnée, et `adresse(nom, **details)` , qui renvoie le nom suivi des détails sous la forme `Ndeye Sarr | ville : Thiès | quartier : Grand Thiès` . (Section 3.)
- ★★ **Exercice 3 : trois compréhensions.** Avec les mots wolof `teranga, jamm, dekk, suba, xarit, ndox, liggeey` : le dictionnaire mot : longueur des mots de plus de 4 lettres ; l'ensemble des premières lettres ; puis une table de multiplication de 3 lignes et 5 colonnes, en une compréhension imbriquée (`[[i * j for j in ...] for i in ...]`). (Section 6.)
- ★★ **Exercice 4 : des règles en paramètre.** Écrivez `appliquer_remise(prix, regle)` et un dictionnaire de trois règles écrites en `lambda` : aucune remise, 10 % de remise (arrondi), 500 F de moins sans descendre sous 0. Testez sur 4 000 F. Puis triez les étudiants `("Awa", 19, 14.5)` , `("Moussa", 21, 12.0)` , `("Khady", 20, 16.25)` (nom, âge, moyenne) par moyenne décroissante, puis par âge. (Section 5.)
- ★★★ **Exercice 5 : aplatir.** Écrivez une fonction récursive `aplatir(liste)` qui transforme `[1, [2, [3, 4], 5], [[6]], 7]` en `[1, 2, 3, 4, 5, 6, 7]` , et une fonction récursive `profondeur(liste)` qui donne le nombre de niveaux d'imbrication (3 ici ; 1 pour `[]` et pour `[1, 2]`). Indice pour `profondeur` : `max(..., default=0)` . (Section 7.)

Atelier 4 : filtres et rapports à la demande

Le travail principal du chapitre : la direction de Yonnee Express pose sans cesse de nouvelles questions (« les retards à Pikine ? », « les colis lourds d'Awa en 2026 ? », « la durée moyenne par livreur ? »). Plutôt qu'un programme par question, vous construisez une petite **boîte à outils** : des fabriques de filtres combinables, une fonction de rapport réglable, et une fonction récursive qui regroupe les livraisons sur autant de niveaux qu'on veut. L'énoncé vous est fourni séparément.

A retenir

- Paramètres par défaut **après** les obligatoires ; arguments **nommés** pour la lisibilité ; jamais de liste ou de dictionnaire par défaut (`None` , puis créer dans la fonction).
- `*args` reçoit un tuple, `**kwargs` un dictionnaire ; à l'appel, `f(*liste)` et `f(**dico)` déballent.
- Une variable créée dans une fonction est **locale** ; ordre de recherche LEGB ; ne pas modifier de variable globale depuis une fonction.
- Une fonction est une **valeur** : on la passe sans parenthèses ; une fonction peut en fabriquer une autre (fermeture).
- `lambda x: expression` pour une petite fonction ; `map` , `filter` ; souvent plus lisible : une compréhension.
- Compréhensions : `[... for ... if ...]` , `{c: v for ...}` , `{... for ...}` ; `if ... else` **avant** le `for` pour choisir, `if` **après** pour filtrer.
- Récursivité : un **cas de base** qui s'arrête, un **cas récursif** plus petit ; idéale pour les structures imbriquées.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`TypeError: f() missing 1 required positional argument: '...'`	Un argument obligatoire manque	Le donner, ou lui mettre une valeur par défaut
`TypeError: f() takes from 2 to 4 positional arguments but 5 were given`	Trop d'arguments	Vérifier la définition ; utiliser des arguments nommés
`TypeError: f() got an unexpected keyword argument '...'`	Nom de paramètre mal écrit	Vérifier le nom dans la définition
`SyntaxError: parameter without a default follows parameter with a default`	Un paramètre obligatoire placé après un paramètre par défaut	Mettre les obligatoires en premier
Une liste par défaut « se souvient » des appels précédents	Valeur par défaut modifiable	`liste=None` , puis `if liste is None: liste = []`
`UnboundLocalError: cannot access local variable '...'`	Modification d'une variable globale dans une fonction	Passer la valeur en paramètre et la renvoyer
`NameError: name '...' is not defined` (hors d'une fonction)	Variable locale utilisée à l'extérieur	Renvoyer la valeur avec `return`
`TypeError: ... missing 1 required positional argument` en passant une fonction	Fonction passée **avec** parenthèses	La passer sans parenthèses : `appliquer(liste, est_lourd)`
`RecursionError: maximum recursion depth exceeded`	Pas de cas de base, ou le problème ne rapetisse pas	Vérifier le cas de base et que l'appel récursif progresse vers lui

Pour aller plus loin

- Le tutoriel officiel, « Davantage sur la définition des fonctions » (en français) : https://docs.python.org/fr/3/tutorial/controlflow.html#more-on-defining-functions
- Les compréhensions de listes, dans le tutoriel officiel : https://docs.python.org/fr/3/tutorial/datastructures.html#list-comprehensions
- Le guide de programmation fonctionnelle de Python : https://docs.python.org/fr/3/howto/functional.html